Video Transcript

Video 1: Introduction (04:10)

So, this week we're going to be looking at really big data. We've been looking at files that are a few 1,000 lines long, and we've been reading in CSV and JSON formats. This week we're going to read a file that's over 100 million lines long and we're going to see that pandas takes quite a time to read a file that large. And so, we're going to look at a couple of other formats, Parquet, and Feather, and HDF5 are other formats that are actually faster. And we're also going to look at our machines in some detail because our machines these days have multiple cores and we can run lots of processes and threads on those cores. If you look at your laptop and see how many threads it's running; I looked at mine, it was running around 3,000 threads.

So, we need to understand the difference between cores, threads, and processes, and we need to understand how to exploit those that we can run our programs in parallel. So, we're going to use DASK, which is a Python library which allows us to do things in parallel. We're going to look at, first, writing 20 files at a time. And, then we're going to look at how can we read back those files in parallel. We're going to explore DASK, D-A-S-K and we're also going to look at a library called Graphviz, which shows us our execution graph, how our functions are going to be called and executed. Today, most computation is done in parallel. We're taking advantage of machines with 64 cores, 128 cores in some cases.

So, we can do a lot in parallel, and we probably need to do that. So, after we've looked at big files, we'll be looking at really big files, files that are too large for us to actually read into memory. This comes under the heading of streaming data. Often if we're, for example, monitoring mouse clicks on our website, we might be getting a billion events a day being streamed at us and we need to develop a strategy for handling streaming data. We'll start by looking at web sockets. So far, we've been using HTTP and that has a request response built into it, that it's not as if either party can send messages to the other. One party makes a request to the web server and the web server responds. Now, in the case of web sockets, either party can communicate and they can communicate in parallel.

© MIT xPRO® 2024. All rights reserved.

So, while you're sending me data, I could be sending you data. So, web sockets are extremely efficient when it comes to handling streaming data. And we're going to do an exercise where you're going to look at how web sockets handle chunks of data and how we implement on top of those TCP packets, the idea of a message. So, we're going to handcraft our own messages using web sockets. And, this will set us up then for next week, we're going to look at streaming data, MQTT, and Kafka. But for now, let's take a look at really large files and setting up web sockets.

© MIT xPRO® 2024. All rights reserved.